

TTT4265 – Elektronisk systemdesign og -analyse II

(Nummereringen fortsetter fra ESDA I)

Øving 17 2025

Innleveringsfrist mandag 11. november klokka 23:59

(Med oppkobling)

I denne øvingen skal du få implementere en kodelås på FPGA. Du vil få erfaring med implementasjon og design av større digitale systemer og bruk av tilstandsmaskiner i praksis.

Hvordan lure skurken?

Sørgelig men sant: Det finnes skurker som prøver å lure oss. Derfor må vi lure skurken. En vanlig måte å gjøre det på i dag er ved hjelp av PIN-koder som bare eieren kjenner til. Vi bruker dem daglig i betalingsautomater og når vi skal åpne en dør med elektronisk lås.

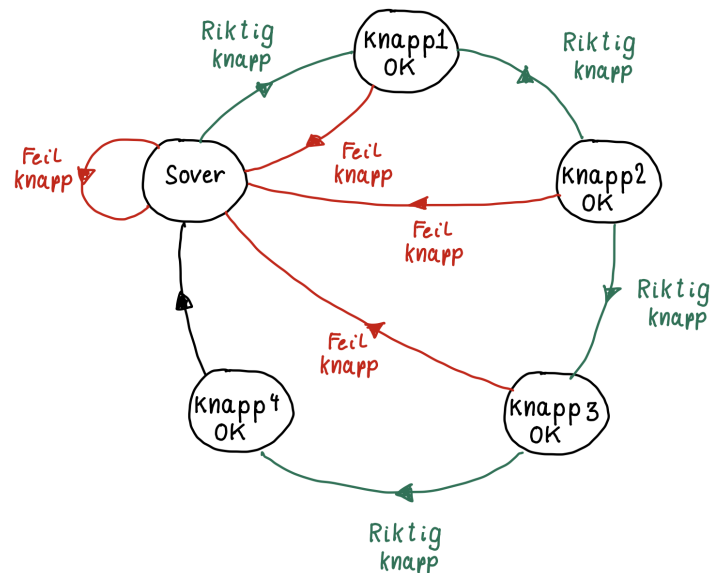
For å vise at du ikke er skurk, må du altså trykke inn fire bestemte siffer i en bestemt rekkefølge. I denne øvingen skal vi se på hvordan elektronikken i en slik elektronisk dørlås (eller en minibank) kan lages.

Situasjonen er altså slik at den som skal trykke, har valget mellom et visst antall knapper (typisk 10 stykker). For å åpne låsen må de riktige knappene trykkes, og de må trykkes i riktig rekkefølge. Det å detektere hvilken knapp som er trykket, er en relativt enkel sak. Det å sjekke den riktige rekkefølgen, er litt verre.

Når det er snakk om at hendelser skal skje i en bestemt rekkefølge, er det ett spesielt hjelpemiddel som alltid er aktuelt: *Tilstandsmaskiner*.

Tilstandsmaskin for en kodelås

Vi tenker oss en tilstandsmaskin som skifter tilstand hver gang en knapp trykkes. Et tilstandsdiagram for en slik situasjon er vist i figur 1.



Figur 1: Tilstandsdiagram for kodelås hvor fire påfølgende knappetrykk kreves,

I utgangspunktet “sover” maskinen. Det vil si at den passivt venter på at en av de mulige knappene trykkes. Dersom knappen for det riktige første siffer i koden trykkes, går maskinen til tilstanden **Knapp 1 OK**. I motsatt fall forblir maskinen i sove-tilstanden. Kanskje er det en skurk som trykker, og som tilfeldigvis har trykket den riktige knapp 1. Dersom vedkommende så trykker enda en knapp, vil den mest sannsynlig være feil, maskinen går tilbake til sove-tilstanden, og skurken må starte på nytt. Derimot vil den som kjenner koden, trykke riktig og gå til **Knapp 2 OK**. Videre vil hvert riktig trykk bringe trykkeren¹ videre mot tilstand **Knapp 4 OK**. Hver gang feil knapp trykkes, går maskinen tilbake til sove-tilstanden, og trykkeren må starte på nytt. I tilstand **Knapp 4 OK** åpnes døren, og maksinen går tilbake til sove-tilstanden.

Konkretisering

Så langt det prinsipielle. Vi skal nå se på hvordan en kodelås kan implementeres ved hjelp av FPGA-kortet *Go Board*. Dette har fire knapper (SW1, SW2, SW3 og SW4) som vi kaller henholdsvis A, B, C og D. Vår løsning baseres på disse knappene i stedet for de ti knappene på et numerisk tastatur. Dette gjør det *litt* enklere for skurken, men det skal likevel godt gjøres å tippe den riktige sekvensen av fire knappetrykk.

¹Siden vi ikke vet om den som trykker er skurk eller ikke, kaller vi vedkommende generelt “trykkeren”.

Vårt kodelås-design kan deles i tre delsystemer, som vist i figur 2.



Figur 2: Tre delsystemer som til sammen utgjør en kodelås.

Overordnet består systemet altså av tre delsystemer: et tastatur for å registrere knappetrykk, en tilstandsmaskin som holder styr på nåværende tilstand og beregner neste basert på knappetrykkene samt en *aktuator* hvis oppførsel styres av hvorvidt kodelåsen er åpen eller ikke.

Tastaturet leverer signalene A, B, C og D som indikerer at den respektive knappen er trykket. Disse signalene sendes til tilstandsmaskinen. Denne gir i sin tur signalet L til aktuatoren som indikerer at døren er låst. Når L går lav åpner aktuatoren låsen.

For vårt bruk kan aktuatoren bestå av to lysdioder med tilhørende logikk. En grønn LED indikerer at døren er åpen, en rød at den er låst.

Hva med klokken?

Som vanlig ønsker vi å lage et synkront system. Det vil si at systemet inneholder et klokkesignal. Dette forteller *når* systemet *kan* skifte tilstand, men ikke *at* en tilstandsending nødvendigvis skjer. Dette kommer vi tilbake til snart.

Oppgave 1 Tastaturet

Tastaturet har altså som oppgave å levere de fire signalene nevnt ovenfor. Vi antar at hver knapp fungerer som en bryter med en nedtrekksmotstand, slik at den normalt leverer verdien 0 bortsett fra når den er holdt nede, hvor verdien er 1.

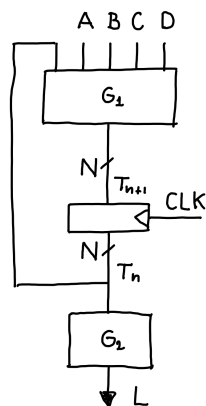
I første omgang høres oppgaven enkel ut, men etter å ha tenkt oss litt om, finner vi et mulig problem: Anta at trykkeren trykker på knappen A og holder den inne i for eksempel 3 klokkeperioder. Hvordan skal systemet skille denne situasjonen fra en situasjon hvor knappen trykkes tre ganger etter hverandre? Dette fant vi en løsning på i øving 16, hvor du implementerte en tastaturmodul for én knapp.

a) Ta utgangspunkt i din egen implementasjon av tastaturmodulen (eller den fra løsningsforslaget til øving 16) slik at du kan bruke knappene SW1 - SW4 på FPGA-kortet til å generere signalene A, B, C og D som trengs til kodelåsen. Test at knappetrykk registreres

korrekt for hver av knappene ved hjelp av logikkanalysatoren til Analog Discovery 2. Husk at du kan importere tastaturmodulen du allerede har laget som en egen blokk i designet ditt.

Oppgave 2 Tilstandsmaskinen

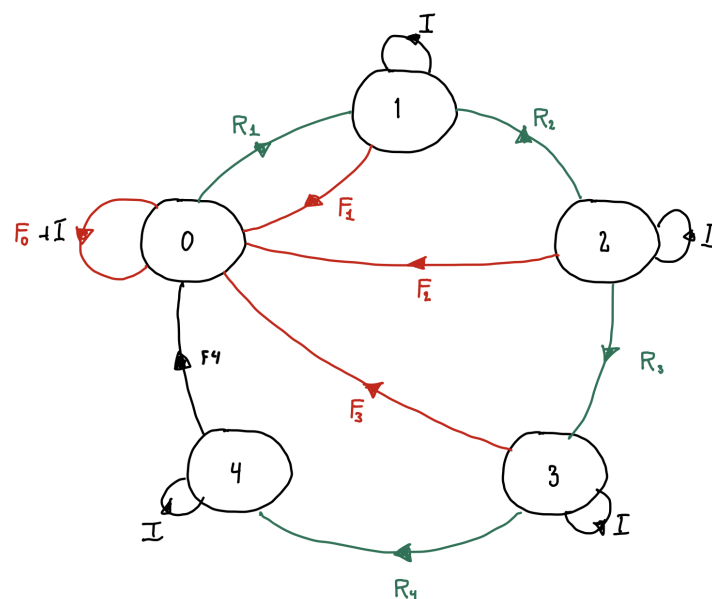
Som alle tilstandsmaskiner har vår i utgangspunktet en enkel struktur. Denne er skissert i figur 3.



Figur 3: Overordnet struktur for tilstandsmaskinen

Sentralt i en hver tilstandsmaskin står *tilstandsregisteret*. Dette er et N bits register som holder på (den nåværende) tilstanden T_n . Basert på denne, samt inngangene A, B, C og D fra tastaturet, beregnes neste tilstand T_{n+1} i blokken merket G_1 . Når slutttilstanden er oppnådd, genereres signalet L av blokken G_2 som sendes til aktuatoren.

- a) Hvor mange bit N trengs i vårt tilfelle?
- b) Vi antar nå at tilstandsvariabelen T antar verdiene 0, 1, 2, 3 og 4 tilsvarende de fem tilstandene i figur 1. Vi kan da tegne et nytt tilstandsdiagram som vist i figur 4.



Figur 4: Tilstandsdiagram med nummererte tilstander

Her har vi også definert logiske variable R_n for “Riktig knapp” og F_n for “Feil knapp” som angir betingelsene for å skifte mellom tilstandene skal være. Vi har også indikert en variabel I . Denne står for “Ingen knapp trykket” og tilsier at maskinen blir stående i sin nåværende tilstand.

Fra nå av og i resten av øvingen, antar vi at den riktige koden skal være “ABBA”. Videre skal systemet returnere til tilstand 0 når hvilken som helst av knappene blir trykket i tilstand 4. Denne transisjonen har fått navnet F_4 , men er strengt tatt ikke et feiltrykk.

I tabell 1 har vi vist hvordan variablene R_1 og F_0 kan defineres utfra hvilke knapper som til enhver tid er trykket.

Tabell 1: Variabler for skifting mellom tilstander

Variabel	Logisk uttrykk
F_0	$B + C + D$
R_1	$A\bar{B}\bar{C}\bar{D}$
F_1	
R_2	
F_2	
R_3	
F_3	
R_4	
F_4	

Fyll ut resten av tabellen.

c) Formuler et logisk uttrykk for variabelen I i figur 4.

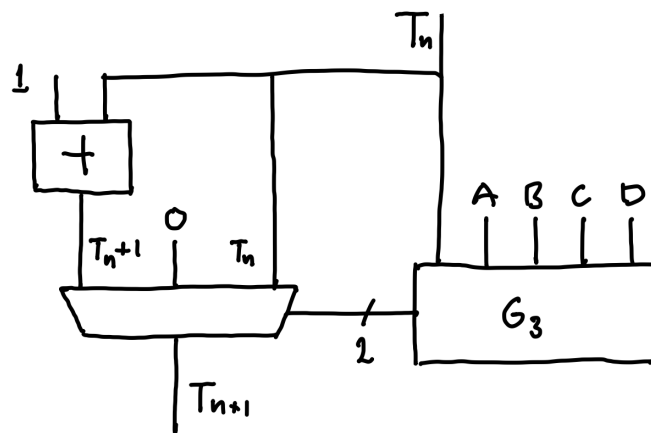
Oppgave 3 Neste-tilstands-funksjonen

Dersom tilstandsregisteret er hjertet av tilstandsmaskinen, må neste-tilstands-funksjonen (som vi har benevnt med G_1 i figur 3) kalles hjernen. Det er nemlig den som avgjør om og når maskinen skal skifte tilstand, altså å beregne neste tilstand utfra nåværende og eventuelle inngangssignaler. Mye av arbeidet er gjort i og med tabell 1, men det gjenstår å lage et fullstendig digitalt design.

Fra figur 4 ser vi at dersom vi er i tilstand T_n , har vi tre muligheter: Enten forblir vi i tilstanden dersom ikke noe skjer, eller vi returnerer til T_0 om trykkeren har trykt feil, eller vi avanserer til tilstand $T_{n+1} = T_n + 1$ dersom riktig knapp er blitt trykket.² Dette kan vi formulere matematisk slik:

$$T_{n+1} = \begin{cases} T_n & \text{dersom } I \\ T_n + 1 & \text{dersom } R_n \\ 0 & \text{dersom } F_n \end{cases}$$

Vi kan da strukturere selve logikken som vist i figur 5.



Figur 5: Logikk for neste-tilstands-funksjon, G_1 .

Her velges en av de tre mulige neste-tilstandene ved hjelp av en multiplekser. En mulig konfigurasjon for oppførselen til multiplekseren er beskrevet i tabell 2.

²Her ser vi forresten hvor viktig det er med presis notasjon. T_{n+1} og $T_n + 1$ betyr to forskjellige ting.

Tabell 2: Sannhetstabell for multiplekser

T_{n+1}	SEL_1	SEL_0
T_n	0	0
$T_n + 1$	0	1
0	1	0
0	1	1

Her er signalene SEL_1 og SEL_0 styringssignalene til multiplekseren. Blokken G_3 brukes til å bestemme disse og styre multiplekseren basert på hvilke av de fire knappene som er trykket inn og nåværende tilstand T_n .

- a) Sett opp logiske uttrykk og/eller sannhetstabeller for signalene SEL_1 og SEL_0 . Bruk gjerne variablene fra tabell 1 i uttrykkene.
- b) Bruk uttrykkene du har funnet og design innholdet i blokken G_3 . Det kan være lurt å lage en egen kodeblokk for å dekode tilstandsvariabelen T_n slik at hver tilstand kan representeres med et eget signal.
- c) Implementer tilstandsmaskinen på FPGA og test at den virker. I designprosjekt 2 ESDA I bør du ha implementert MUXer og adderere av samme type som du trenger her, så gjenbruk gjerne disse. For å teste at modulen virker kan det være lurt å bruke LEDene på FPGA-kortet til å indikere nåværende tilstand. Eventuelt kan du monitorere systemet med logikkanalysatoren. Husk å bruke tastaturmodulen du laget i oppgave 1 for å registrere knappetrykk.
- d) Implementer aktuatorfunksjonen i blokk G_2 , for eksempel i form av to lysdioder som nevnt i introduksjonen.
- e) Hva skjer dersom noen prøver å trykke mer enn én knapp samtidig?